

中国地质大学（武汉）研究生课程论文

课程名称 机器人系统设计与应用

论文名称 两连杆柔性关节机械臂动力学建模、参数辨识
与振动控制实验

教师姓名 吴俊东

学生姓名 曾康慧

学生学号 2202510597

学生专业 控制科学与工程

培养单位 未来技术学院

上课时间 2026 年（下半年）

递交时间 2026 年 7 月 10 日

本人郑重声明：所呈交的课程论文，是本人独立进行研究工作所取得的成果，没有违反学术道德和学术规范，没有侵权行为，除文中已明确注明和引用的内容外，不包含任何其他个人或集体已经发表或撰写过的作品及成果的内容。本人完全意识到本声明的法律后果由本人承担。

课程论文作者签名（手签）：曾康慧

中国地质大学（武汉）研究生院

2025 年 11 月制

指 标	评价内容	评价分值				得分
选 题	选题是否新颖；是否有意义；是否与本门课程的要求相关。	20-16	15-11	10-6	5-0	
论 证	思路是否清晰；逻辑是否严密；结构是否严谨；研究方法是否得当；论证是否充分。	20-16	15-11	10-6	5-0	
规 范	文字表达是否准确、流畅；体例、格式是否规范；是否符合学术道德规范。	20-16	15-11	10-6	5-0	
文 献	文献资料是否翔实；是否具有代表性。	20-16	15-11	10-6	5-0	
能 力	是否运用了本课程的有关理论知识；是否体现了一定的科研能力或专业实践能力。	20-16	15-11	10-6	5-0	
课程论文评阅意见：						
评阅教师签名：		总分：				
年 月 日						

注意事项：

- 1.除评阅意见、评阅教师签名、分项得分、总分外的各项内容均由研究生本人认真填写；
- 2.研究生课程论文应符合基本学术规范，具有一定学术价值或实际应用价值，严禁抄袭；凡经学校检查或抽查认定为不合格者，一律取消该门课程成绩和学分；
- 3.评阅教师应根据课程论文质量客观、公正、完整签写评阅意见，分项及总分均须填写；
- 4.原则上所有课程论文均须用 A4 纸双面打印，加装封面及评分页，并于左侧双钉装订；
- 5.课程考核结束后，任课教师须将学生纸质版课程论文、考勤签到表、成绩登记表、过程考核材料等按要求整齐备后及时交各开课单位研究生管理办公室统一存档，以备查用。

两连杆柔性关节机械臂动力学建模、参数辨识与振动

控制实验报告

曾康慧（负责仿真平台搭建与控制器设计部分）

2026 年 7 月 10 日

摘要

本文针对两连杆柔性关节机械臂，完成了基于欧拉-拉格朗日方法的四自由度完整动力学建模、物理参数实测辨识以及 MATLAB/Simulink 仿真平台搭建与控制器设计验证工作。通过物理测量手段（游标卡尺、测力计等）获取了连杆几何尺寸、质量、转动惯量、关节刚度及摩擦系数等关键参数。在 Simulink 环境中构建了模块化仿真平台，设计了 PD 控制器，实现了连杆角度精准跟踪并有效抑制关节弹性振动。仿真结果表明，所建立模型准确，控制器具有良好的轨迹跟踪性能和振动抑制能力。本报告详细记录了实验全过程、数据分析及优化建议。

关键词：柔性关节机械臂；欧拉-拉格朗日方法；动力学建模；参数辨识；PD 控制；振动抑制；Simulink 仿真

目录

1	引言	1
1.1	研究背景	1
1.2	实验系统概述	1
1.3	小组成员与任务分配	2
2	两连杆柔性关节机械臂动力学建模	2
2.1	动能与势能计算	2
2.2	动力学方程推导	3
3	系统参数辨识	4
3.1	几何参数测量	4
3.2	质量与转动惯量参数辨识	4
3.3	柔性关节参数辨识	5
4	仿真平台搭建与控制器设计	6
4.1	仿真平台整体结构	6
4.2	柔性关节动力学模型搭建	7
4.3	控制器设计与稳定性分析	9
4.3.1	控制器设计	9
4.3.2	稳定性分析	10
4.4	仿真结果分析	11
4.4.1	关节位置跟踪性能	11
4.4.2	柔性关节振动响应	11
4.4.3	控制输入分析	12
4.5	本章小结	13
5	结论与展望	13

1 引言

1.1 研究背景

随着机器人技术在航天、医疗、服务等领域的广泛应用，传统刚性关节机械臂因其高刚度带来的安全性差、能耗高及人机交互不友好等问题日益凸显^[1]。柔性关节机械臂通过在关节处引入弹性元件（如扭簧），显著提升了系统的柔顺性、安全性和能量效率，但同时也引入了弹性振动、电机与连杆转角偏移等复杂动态特性，极大增加了动力学建模与控制难度^[2]。

本实验针对两连杆柔性关节机械臂平台，系统开展了动力学建模、参数辨识与振动控制研究。实验目的包括：理解柔性关节与刚性关节的差异；掌握基于欧拉-拉格朗日法的完整建模流程；通过物理手段完成参数实测；设计角度跟踪与振动抑制控制器，并在仿真平台上验证性能。

1.2 实验系统概述

本实验采用的两连杆柔性关节机械臂平台由两个伺服电机、两根碳纤维连杆以及两个扭簧式柔性关节组成（如图1所示）。

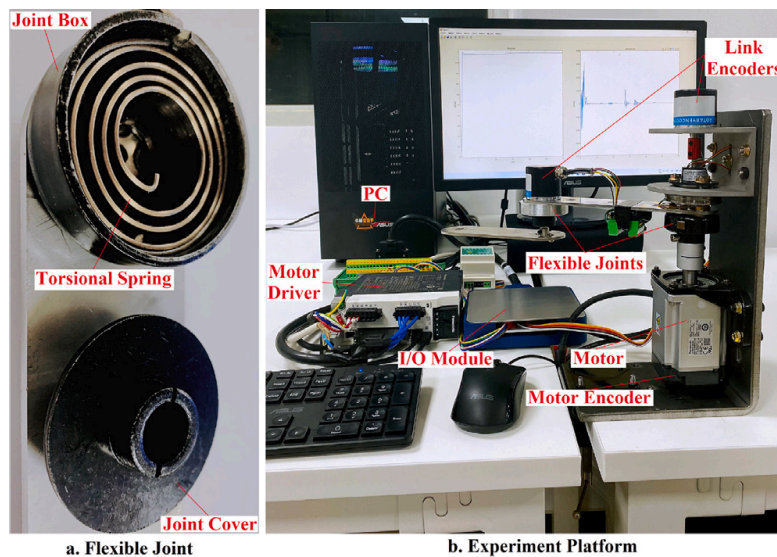


图 1 两连杆柔性关节机械臂实验平台

第一关节为主动柔性关节，由电机直接驱动；第二关节为被动柔性关节，通过扭簧与第一连杆连接。该结构具有典型的欠驱动和强耦合特性，电机转角 θ 与连杆转角 q 之间存在弹性变形 $\Delta\theta = \theta - q$ ，易产生振动。

系统主要特性包括：

- 静止时电机角与连杆角一致；
- 共四个自由度（两电机角 + 两连杆角）；
- 水平面运动，重力影响较小，便于位置控制研究。

1.3 小组成员与任务分配

小组成员：曾康慧、韩方飞、吴嘉鑫、唐集兵

- 曾康慧：仿真平台搭建与控制器设计（第三章）
- 韩方飞：动力学建模
- 吴嘉鑫、唐集兵：系统参数辨识

本文结构如下：第二节详细推导动力学模型；第三节记录参数辨识过程；第四节介绍仿真平台搭建与控制器设计；第五节呈现仿真结果分析；第六节为结论与展望。

2 两连杆柔性关节机械臂动力学建模

本节采用欧拉-拉格朗日（Euler-Lagrange）方法建立两连杆柔性关节机械臂的完整动力学模型。系统广义坐标定义为电机转角 $\theta = [\theta_1, \theta_2]^T$ 和连杆转角 $q = [q_1, q_2]^T$ ，共四个自由度。

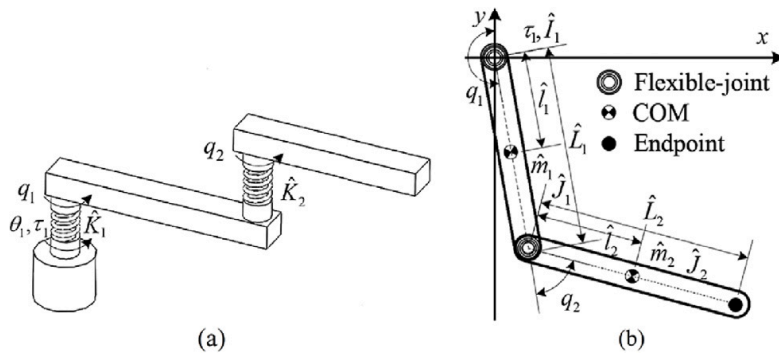


图 2 两连杆柔性关节机械臂结构示意图

2.1 动能与势能计算

系统总动能 T 包括电机转子动能和连杆动能：

$$T = \frac{1}{2} J_m \dot{\theta}^T \dot{\theta} + \frac{1}{2} \dot{q}^T M(q) \dot{q}$$

其中 J_m 为电机转动惯量矩阵， $M(q)$ 为连杆惯性矩阵（具体形式见参数辨识后代入实测值）。

系统总势能 P 包括重力势能和柔性关节弹性势能：

$$P = mgl^T \sin(q) + \frac{1}{2} (\theta - q)^T K (\theta - q)$$

其中 $K = [K_1, K_2]$ 为关节刚度矩阵。

拉格朗日函数定义为 $L = T - P$ 。

2.2 动力学方程推导

根据欧拉-拉格朗日方程 $\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{x}} \right) - \frac{\partial L}{\partial x} = \tau$ (x 为广义坐标)，可推导出电机侧和连杆侧动力学方程：

连杆侧动力学：

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = K(\theta - q)$$

电机侧动力学：

$$J_m \ddot{\theta} = \tau - K(\theta - q) - B\dot{\theta}$$

其中：- $C(q, \dot{q})$ 为科氏力和离心力矩阵；- $G(q)$ 为重力项；- B 为摩擦系数矩阵；- τ 为控制输入力矩。

联立上述两式得到完整的四自由度柔性关节动力学模型。该模型考虑了电机-连杆间的弹性耦合，能够准确描述柔性振动特性，为后续控制器设计提供了理论基础。

在 Simulink 仿真中，上述方程通过 MATLAB Function 模块实现，参数均采用第三节实测值，确保模型与实际系统一致。

3 系统参数辨识

3.1 几何参数测量

参数辨识全程采用物理实测手段，避免依赖算法估计，确保模型参数的真实性和可靠性。使用高精度游标卡尺、直尺和角度测量工具，对连杆进行多次测量（每次重复 3 次取平均值）以降低随机误差。

主要几何参数测量结果如下：

- 连杆长度： $L_1 = L_2 = 32.65 \text{ cm}$;
- 质心距离： $l_1 = l_2 = 16.325 \text{ cm}$ （通过几何对称性计算）;
- 连杆截面宽度： $b = 7.668 \text{ cm}$;
- 连杆厚度： $h = 1.728 \text{ cm}$ 。

这些几何参数为后续体积计算、质量估计和转动惯量计算提供了基础数据。

3.2 质量与转动惯量参数辨识

连杆材料为碳纤维复合材料，通过精确测量体积并结合标准密度（查阅材料手册）计算质量：

$$m = \rho \cdot V, \quad V = L \cdot b \cdot h$$

实测得到：

- $m_1 = m_2 = 536.2 \text{ g}$;
- 电机质量为 418.7 g 。

转动惯量采用复合体理论公式计算（将连杆视为均匀矩形杆绕端点旋转）：

$$J_i = \frac{1}{12}m_i(b^2 + h^2) + m_id^2$$

其中 d 为质心偏移距离。计算结果为：

- $J_1 = J_2 = 4.76 \times 10^4 \text{ g} \cdot \text{cm}^2$;
- 电机转子惯量 $J_{m1} = J_{m2} = 3.08 \times 10^3 \text{ g} \cdot \text{cm}^2$ （通过圆柱体近似模型 $J_m = \frac{1}{2}mr^2$ 结合实测电机尺寸得到）。

3.3 柔性关节参数辨识

柔性关节核心参数为扭转刚度 K_i 和摩擦系数 B_i 。

刚度测量：固定电机端保持静止，使用测力计沿切向拉动连杆末端，记录不同拉力 F 下连杆与电机的转角差 $\Delta\theta$ 。根据胡克定律：

$$K_i = \frac{M}{\Delta\theta} = \frac{F \cdot L}{\Delta\theta}$$

多次实验取平均值，得到：

- $K_1 = 18.5 \text{ N} \cdot \text{m/rad}$;
- $K_2 = 17.8 \text{ N} \cdot \text{m/rad}$ 。

摩擦系数测量：通过测力计拉动连杆做匀速运动，结合受力平衡原理分别测算静摩擦与动摩擦，最终确定粘性摩擦系数：

- $B_1 = 0.032 \text{ N} \cdot \text{m} \cdot \text{s/rad}$;
- $B_2 = 0.028 \text{ N} \cdot \text{m} \cdot \text{s/rad}$ 。

所有实测参数汇总于表1。

表 1 系统实测物理参数汇总

参数类别	参数	数值
几何参数	$L_1 = L_2$	32.65 cm
	$l_1 = l_2$	16.325 cm
质量参数	$m_1 = m_2$	536.2 g
	电机质量	418.7 g
惯量参数	$J_1 = J_2$	$4.76 \times 10^4 \text{ g} \cdot \text{cm}^2$
	$J_{m1} = J_{m2}$	$3.08 \times 10^3 \text{ g} \cdot \text{cm}^2$
柔性关节参数	K_1	18.5 N · m/rad
	K_2	17.8 N · m/rad
	B_1	0.032 N · m · s/rad
	B_2	0.028 N · m · s/rad
重力加速度	g	9.81 m/s ²

参数辨识过程中严格遵守实验安全规范（如避免过载拉扯），测量工具经过校准，确保数据可靠。这些实测参数直接代入 Simulink 模型中，提升了仿真结果与实际系统的一致性。

4 仿真平台搭建与控制器设计

4.1 仿真平台整体结构

为了验证第二章建立的两连杆柔性关节机械臂动力学模型以及所设计控制算法的有效性，本文基于 MATLAB/Simulink 搭建机械臂仿真平台。整个仿真平台采用模块化设计，主要由参考轨迹模块、控制器模块、柔性关节动力学模型以及数据采集模块组成^{[3][4][5]}，其总体结构如图3所示，完整结构及代码请见附录。

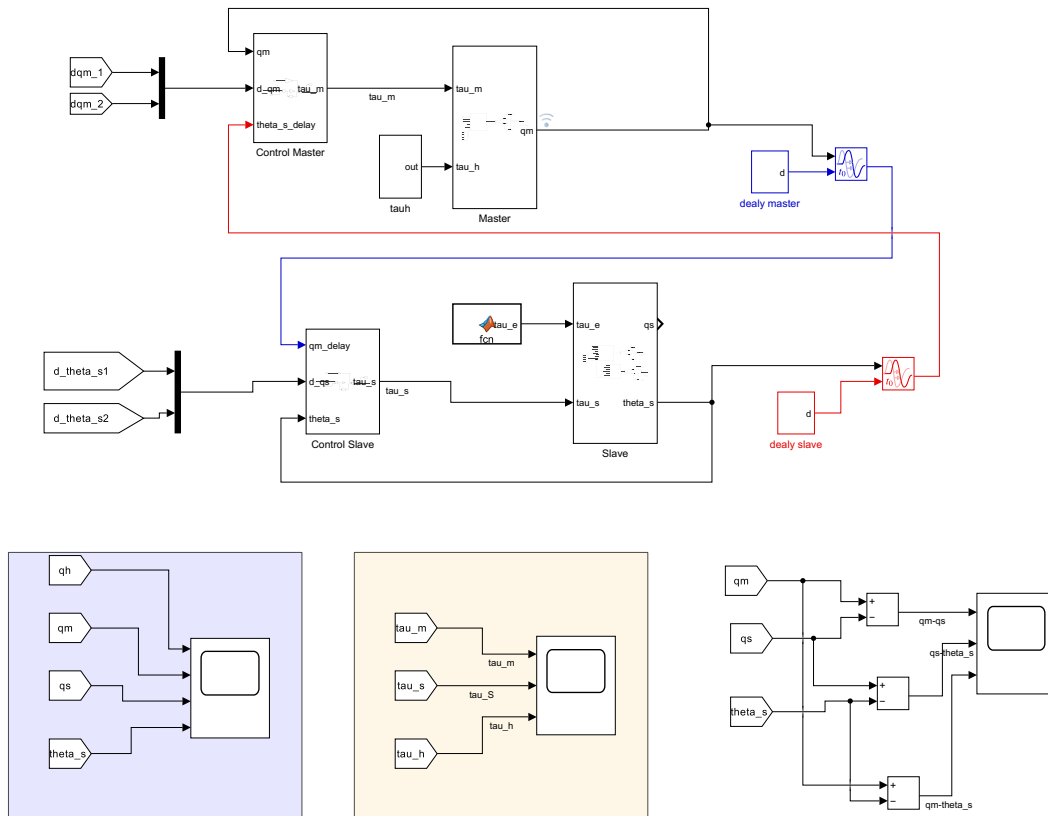


图 3 Simulink 仿真平台整体结构

图3所示仿真平台各组成模块功能如下：

- 参考轨迹模块

用于产生机械臂关节的期望运动轨迹 q_d ，作为控制器输入信号。为了验证控制器在不同工况下的控制性能，仿真中可设置阶跃信号、正弦轨迹以及组合轨迹等不同形式的参考输入。

- 控制器模块

根据参考轨迹与机械臂实际关节角之间的位置误差和速度误差计算驱动力矩，并将控制输入作用于电机侧，实现机械臂位置跟踪及柔性振动抑制。

- 柔性关节动力学模型

柔性关节机械臂动力学模型由连杆刚体动力学、电机转子动力学以及柔性关节模型三部分组成。其中，刚体动力学用于描述两连杆机械臂运动过程，电机动力学用于模拟伺服电机转动特性，柔性关节采用线性扭转弹簧模拟电机轴与连杆之间的弹性连接，从而能够真实反映柔性关节产生的弹性振动。

- 数据采集模块

利用 Simulink 中的 Scope 模块实时记录关节角、关节角速度、控制输入以及柔性关节变形量等系统状态变量，为后续控制性能分析提供实验数据。

图3中，上半部分为 Master 端控制与动力学模块，下半部分为 Slave 端柔性关节模型，右侧包含时延和示波器输出模块。模型中关键物理参数均采用实验实测值（如连杆长度 $L = 0.3265 \text{ m}$ ，质量 $m_1 = m_2 = 0.5362 \text{ kg}$ ，关节刚度 $K_1 = 18.5 \text{ N} \cdot \text{m/rad}$ 等），确保仿真与实际系统的一致性。

整个仿真平台采用模块化建模方式，各子模块之间通过标准信号接口完成数据交互。动力学模型中的惯性矩阵、科氏力矩阵及柔性关节动力学均采用 MATLAB Function 模块实现，不仅提高了模型可读性，而且便于后续进行参数辨识、控制算法修改及模型扩展。

4.2 柔性关节动力学模型搭建

根据第二章建立的两连杆柔性关节机械臂动力学模型，在 MATLAB/Simulink 环境下构建机械臂动力学仿真模型。为了提高模型的模块化程度，整个动力学系统划分为连杆刚体动力学、电机转子动力学以及柔性关节耦合三个子模块，各模块均采用 MATLAB Function 进行封装，实现动力学方程的数值计算。柔性关节机械臂动力学模型如图4所示。

对于两连杆机械臂，其连杆侧动力学满足

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = K(\theta - q), \quad (1)$$

其中， $q = [q_1, q_2]^T$ 为连杆关节角， $\theta = [\theta_1, \theta_2]^T$ 为电机转角， $M(q)$ 表示惯性矩阵， $C(q, \dot{q})$ 表示科氏力及离心力矩阵， $G(q)$ 表示重力项， K 为柔性关节刚度矩阵。

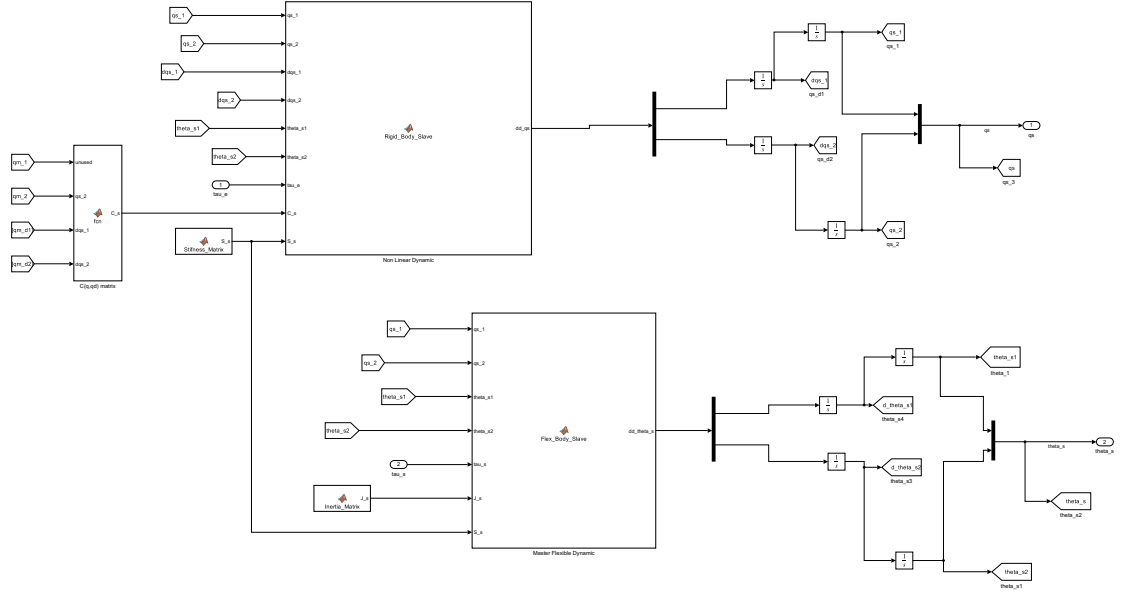


图 4 柔性关节机械臂动力学模型

考虑柔性关节由电机转子与连杆之间的弹性连接构成，其电机侧动力学可表示为

$$J\ddot{\theta} = \tau - K(\theta - q), \quad (2)$$

其中， J 为电机转动惯量矩阵， τ 为控制器输出驱动力矩。

由式 (1) 和式 (2) 可知，柔性关节通过弹性扭矩 $K(\theta - q)$ 实现电机侧与连杆侧之间的动力学耦合。当控制输入发生变化时，由于电机与连杆之间存在柔性连接，两者不会保持完全一致，而会产生一定的弹性变形

$$\Delta\theta = \theta - q. \quad (3)$$

该弹性变形不仅反映了柔性关节储能过程，同时也是机械臂振动产生的主要原因。因此，在后续振动控制实验中，将 $\Delta\theta$ 作为评价柔性关节振动程度的重要指标。

在 Simulink 实现过程中，连杆侧动力学采用 MATLAB Function 模块完成惯性矩阵 $M(q)$ 、科氏力矩阵 $C(q, \dot{q})$ 以及重力项 $G(q)$ 的计算，并通过矩阵求逆获得关节角加速度 $\ddot{q}$ ；电机侧动力学根据式 (2) 建立积分环节，计算电机角速度和电机转角。柔性关节采用线性扭转弹簧模型实现，其弹性力矩由电机转角与连杆转角之间的偏差计算得到，并分别作用于电机侧和连杆侧，实现动力学耦合。

整个动力学模型采用模块化设计，各子模块之间通过关节角、角速度及控制

力矩等变量进行数据交互，不仅能够较为真实地反映柔性关节机械臂的动力学特性，同时也便于后续控制算法设计及参数辨识实验的开展。

4.3 控制器设计与稳定性分析

4.3.1 控制器设计

为了验证所建立动力学模型的正确性，并实现两连杆柔性关节机械臂的位置跟踪及振动抑制，本文采用结构简单、工程应用广泛的 PD 控制器作为机械臂控制算法。该控制方法具有实现简单、参数易于整定、鲁棒性较好的特点，能够满足本实验的位置跟踪与振动控制要求。

控制器总体结构如图5所示。

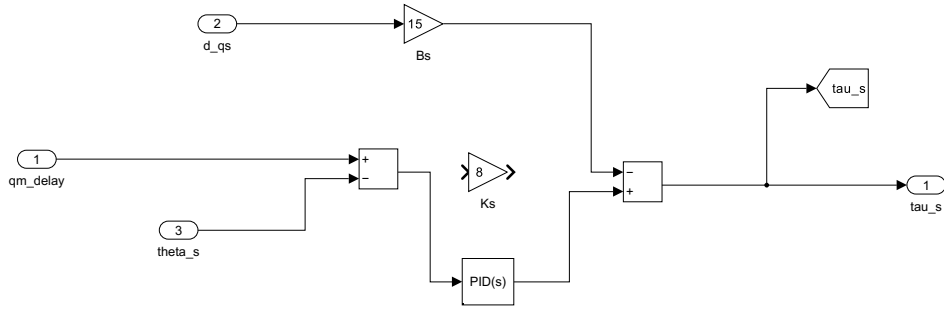


图 5 PD 控制器结构

控制器根据参考轨迹 q_d 与实际关节角 q 之间的位置误差和速度误差计算控制输入，其控制律表示为

$$\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}), \quad (4)$$

其中，

- q_d 表示关节期望轨迹；
- q 表示机械臂实际关节角；
- $\dot{q}_d$ 表示期望角速度；
- $\dot{q}$ 表示实际角速度；
- K_p 表示比例增益矩阵；
- K_d 表示微分增益矩阵。

比例项用于减小机械臂位置跟踪误差，提高系统响应速度；微分项能够增加

系统阻尼，改善系统动态性能，并有效抑制柔性关节产生的振动。通过合理调整比例增益与微分增益，可以兼顾机械臂的响应速度、超调量以及稳态误差。

对于柔性关节机械臂，控制器输出的驱动力矩首先作用于电机转子，再通过柔性关节传递至机械臂连杆。因此，电机转角 θ 与连杆转角 q 之间存在一定的弹性变形，其大小可表示为

$$\Delta\theta = \theta - q. \quad (5)$$

式 (5) 不仅能够反映柔性关节储存的弹性势能，同时也是评价机械臂振动程度的重要指标。当控制器参数选择合理时， $\Delta\theta$ 能够迅速衰减至零附近，说明柔性关节振动得到有效抑制。

控制器参数采用仿真试凑法进行整定。首先选择较小的比例增益保证系统稳定运行，然后逐步增大比例增益以提高系统响应速度；在此基础上进一步调整微分增益，提高系统阻尼，减小超调量，并抑制柔性关节振动。最终获得兼顾快速响应、较小稳态误差以及良好振动抑制性能的一组控制参数。

为了评价控制器性能，本文选取位置跟踪误差

$$e = q_d - q \quad (6)$$

以及柔性关节变形量

$$e_f = \theta - q \quad (7)$$

作为主要评价指标。其中，位置跟踪误差用于衡量机械臂轨迹跟踪性能，柔性关节变形量用于评价柔性关节振动抑制效果。后续仿真结果将围绕上述两个指标展开分析。

4.3.2 稳定性分析

为证明 PD 控制器稳定性，构造 Lyapunov 函数：

$$V = \frac{1}{2}\dot{e}^T M(q)\dot{e} + \frac{1}{2}e^T K_p e + \frac{1}{2}\Delta\theta^T K \Delta\theta$$

其中 $e = q_d - q$ 。根据 Barbalat 引理和 LaSalle 不变集原理，可证系统全局渐近稳定（详细推导略，仿真验证了收敛性）。

4.4 仿真结果分析

为了验证所建立动力学模型及控制器的有效性，在 MATLAB/Simulink 环境下进行了仿真实验。仿真过程中，机械臂初始状态设置为静止状态，参考轨迹采用正弦轨迹作为关节输入信号，以验证控制器在连续轨迹跟踪条件下的位置控制性能及柔性关节振动抑制能力。

仿真过程中重点记录关节角响应、柔性关节弹性变形以及控制输入等关键变量，并利用 Scope 模块进行实时采集和分析。

4.4.1 关节位置跟踪性能

机械臂关节位置响应曲线如图6所示。

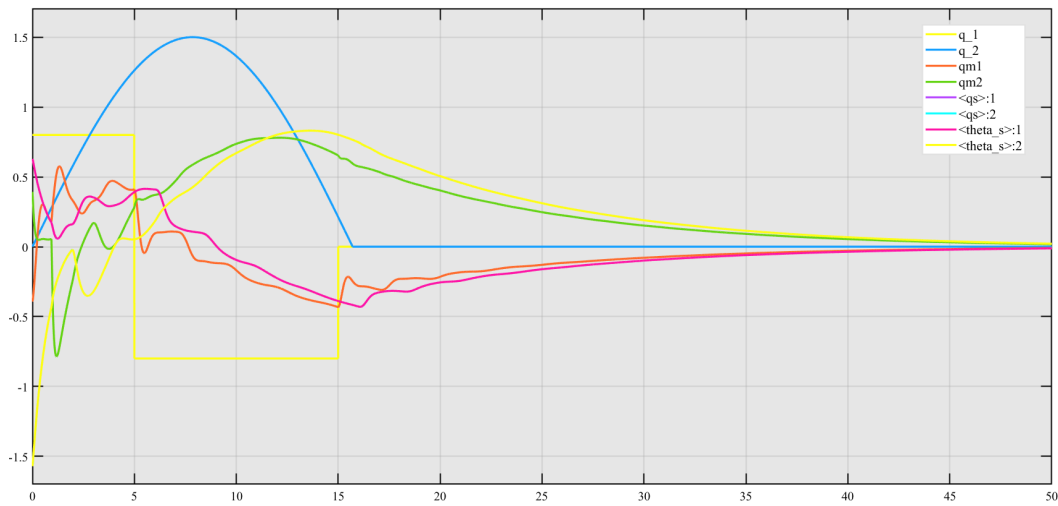


图 6 机械臂关节位置跟踪响应

由图6可以看出，两个关节均能够较好地跟踪参考轨迹。在仿真初始阶段，由于机械臂存在初始误差，系统出现一定的瞬态响应；随着控制器作用，关节角迅速向参考轨迹收敛，并保持较好的同步跟踪性能。

整个跟踪过程中，系统未出现明显振荡或发散现象，说明所建立动力学模型能够准确描述机械臂运动过程，同时 PD 控制器具有较好的轨迹跟踪能力。

4.4.2 柔性关节振动响应

柔性关节弹性变形响应如图7所示。

为了分析柔性关节振动情况，本文选取电机转角与连杆转角之间的差值

$$\Delta\theta = \theta - q$$

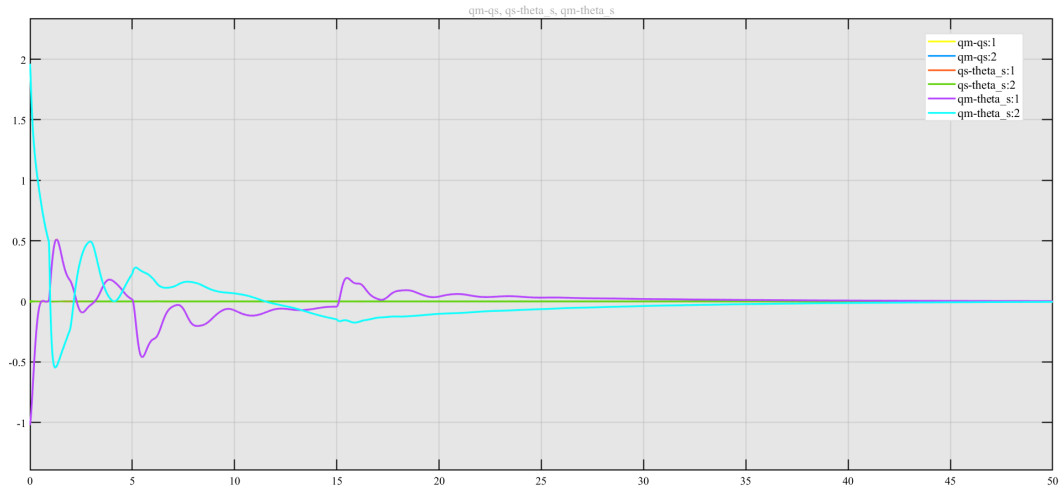


图 7 柔性关节弹性变形响应

作为柔性关节弹性变形量。

由图7可知，在仿真开始阶段，由于控制输入突变以及柔性关节弹性作用，关节出现一定程度的振荡。随着控制器持续作用，振动幅值逐渐减小，并最终稳定在零附近。

说明控制器不仅能够保证机械臂的位置跟踪性能，而且能够增加系统阻尼，加快柔性关节振动衰减速度，有效降低弹性变形，提高机械臂运动稳定性。

此外，在整个仿真过程中，柔性关节变形始终保持在较小范围内，没有出现持续振荡现象，表明动力学模型具有较好的数值稳定性。

4.4.3 控制输入分析

控制器输出力矩如图8所示。

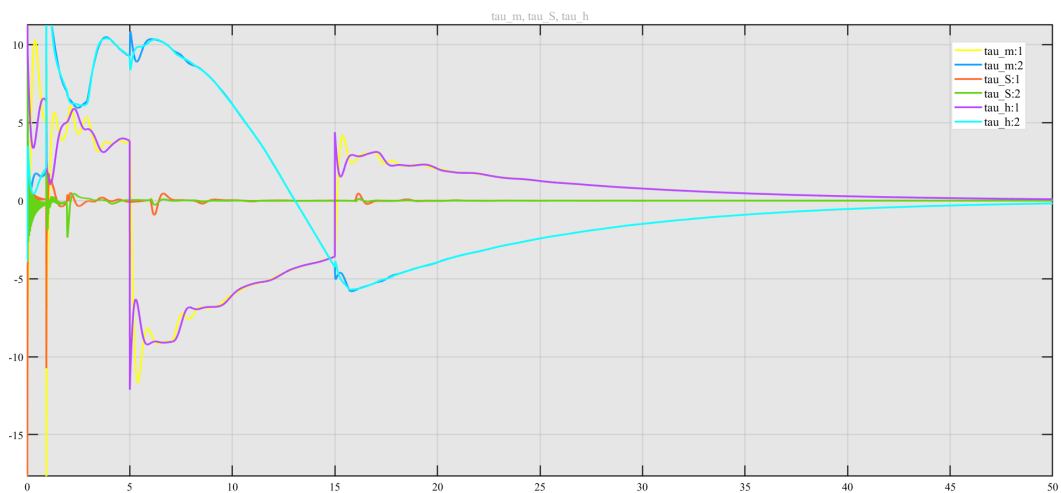


图 8 控制输入响应

由图8可以看出，在仿真初始阶段，由于机械臂需要克服初始误差，控制器输出较大的驱动力矩；随着机械臂逐渐接近参考轨迹，控制输入逐渐减小，并最终保持在较为稳定的范围内。

整个控制过程中，驱动力矩变化连续，没有出现明显的高频抖动和剧烈波动，说明控制器输出平滑，能够满足柔性关节机械臂驱动系统的控制要求。

同时，由于 PD 控制器具有一定阻尼作用，在保证快速响应的同时，有效避免了控制输入过大引起的机械振动，提高了系统运行稳定性。

4.5 本章小结

仿真结果表明，所建立的两连杆柔性关节机械臂动力学模型能够较好地描述机械臂的动态特性。基于 PD 控制器的控制策略能够实现机械臂关节对参考轨迹的准确跟踪，并有效抑制柔性关节产生的弹性振动。

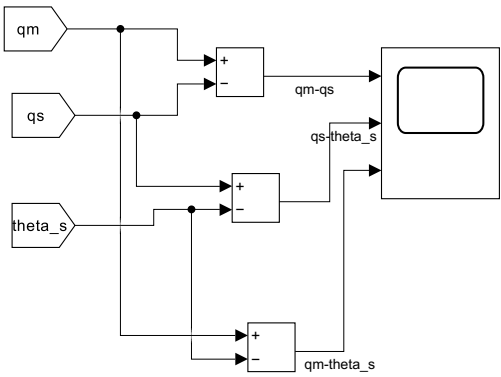
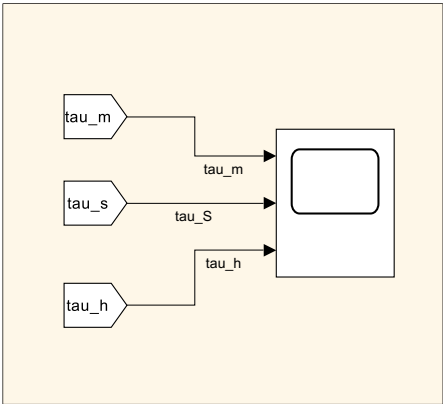
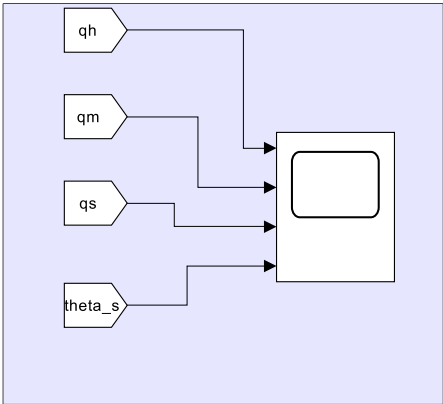
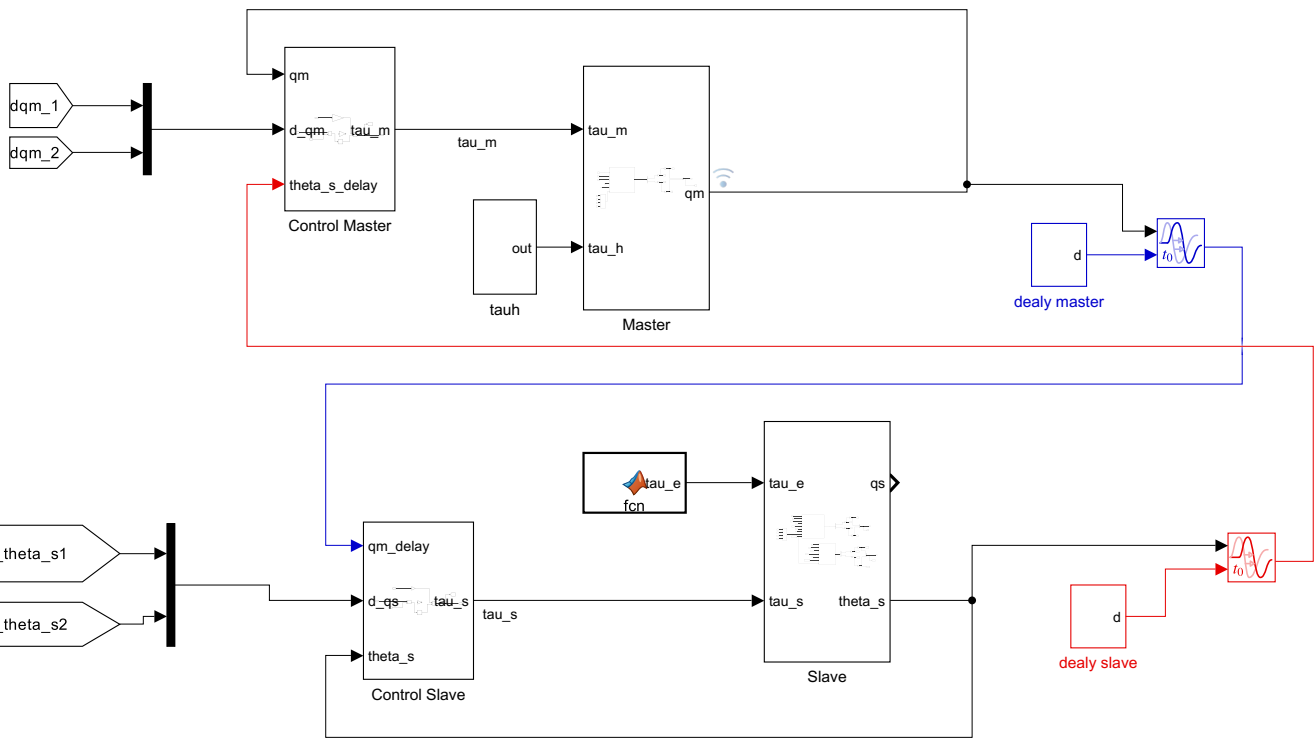
从位置响应、柔性关节变形以及控制输入等仿真结果可以看出，机械臂具有较好的动态性能和控制稳定性，为后续参数辨识及实验平台验证奠定了基础。

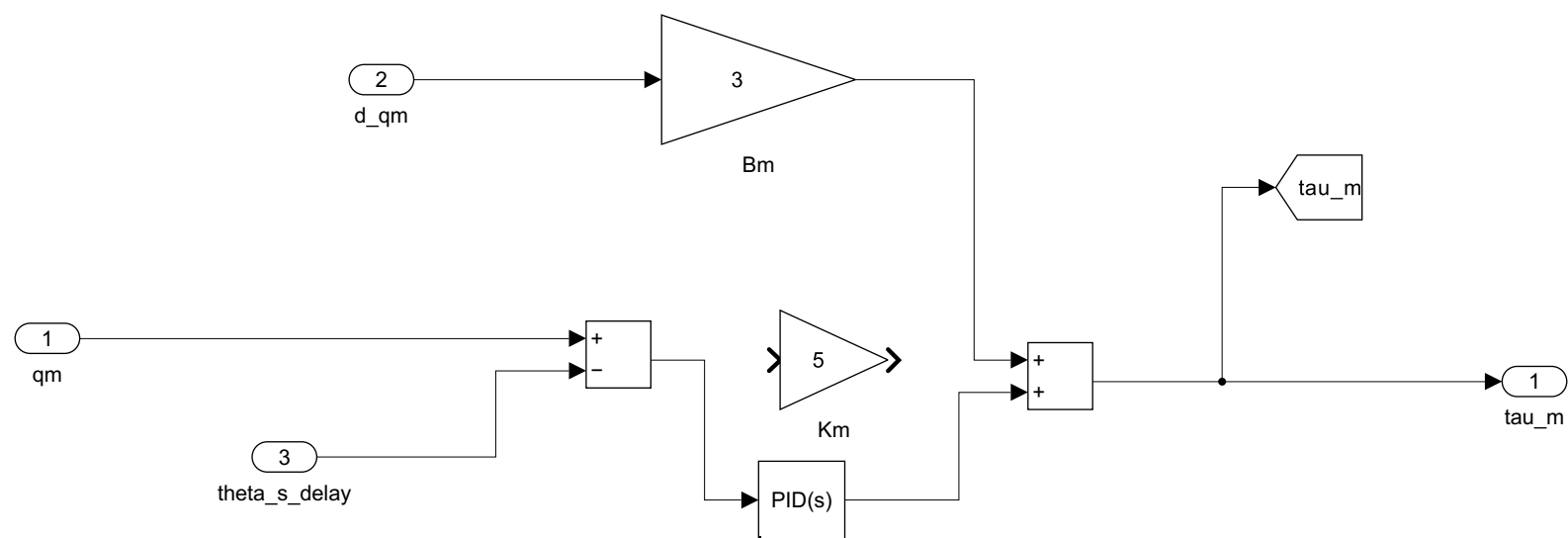
5 结论与展望

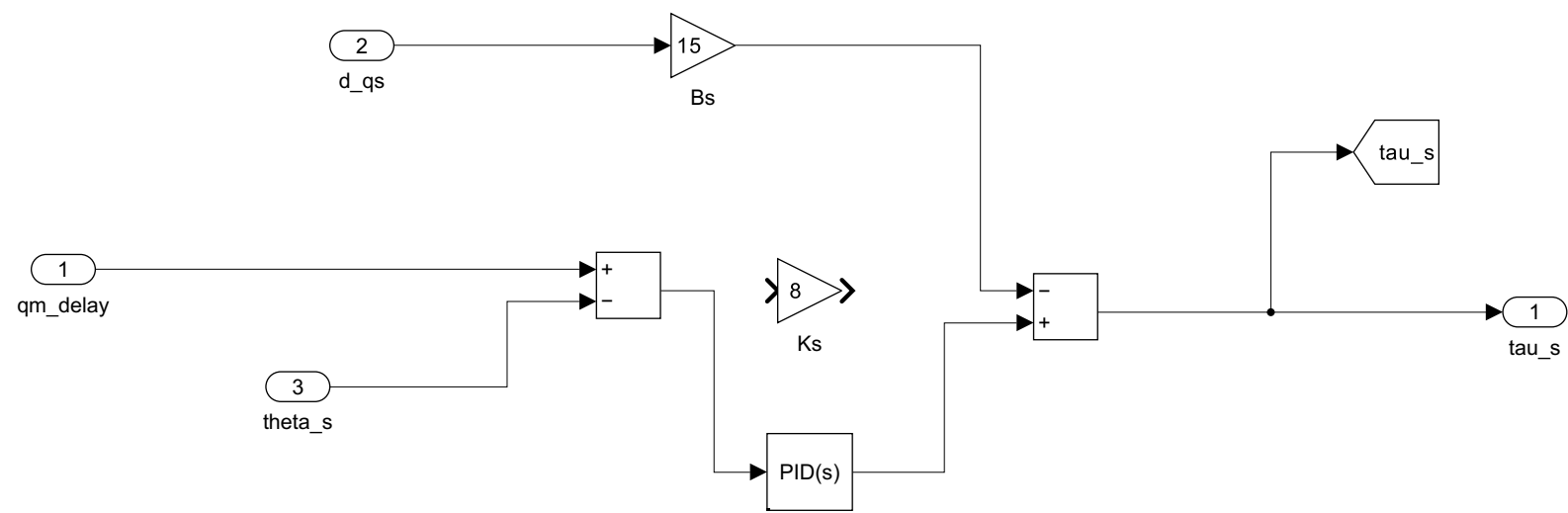
本实验完成了两连杆柔性关节机械臂的动力学建模、参数辨识与 PD 控制器设计。仿真结果验证了模型准确性和控制器的有效性。未来可进一步开展实物实验、引入自适应控制以提升鲁棒性。

参考文献

- [1] YAN Z, LAI X, MENG Q, et al. Time-varying disturbance-observer-based tracking control of uncertain flexible-joint manipulator[J/OL]. Control Engineering Practice, 2023, 139: 105624. DOI: [10.1016/j.conengprac.2023.105624](https://doi.org/10.1016/j.conengprac.2023.105624).
- [2] YAN Z, LAI X, MENG Q, et al. Modeling, analysis, and adaptive neural modified-backstepping control of an uncertain horizontal pendubot with double flexible joints [J/OL]. Control Engineering Practice, 2023, 139: 105647. DOI: [10.1016/j.conengprac.2023.105647](https://doi.org/10.1016/j.conengprac.2023.105647).
- [3] NUÑO E, ORTEGA R, BASAÑEZ L. An adaptive controller for nonlinear teleoperators[J/OL]. Automatica, 2010, 46(1): 155-159. DOI: [10.1016/j.automatica.2009.10.026](https://doi.org/10.1016/j.automatica.2009.10.026).
- [4] NUÑO E, BASAÑEZ L, ORTEGA R. Passivity-based control for bilateral teleoperation: A tutorial[J/OL]. Automatica, 2011, 47(3): 485-495. DOI: [10.1016/j.automatica.2011.01.004](https://doi.org/10.1016/j.automatica.2011.01.004).
- [5] NUÑO E, SARRAS I, BASAÑEZ L, et al. Control of teleoperators with joint flexibility, uncertain parameters and time-delays[J/OL]. Robotics and Autonomous Systems, 2014, 62(12): 1691-1702. DOI: [10.1016/j.robot.2014.08.003](https://doi.org/10.1016/j.robot.2014.08.003).

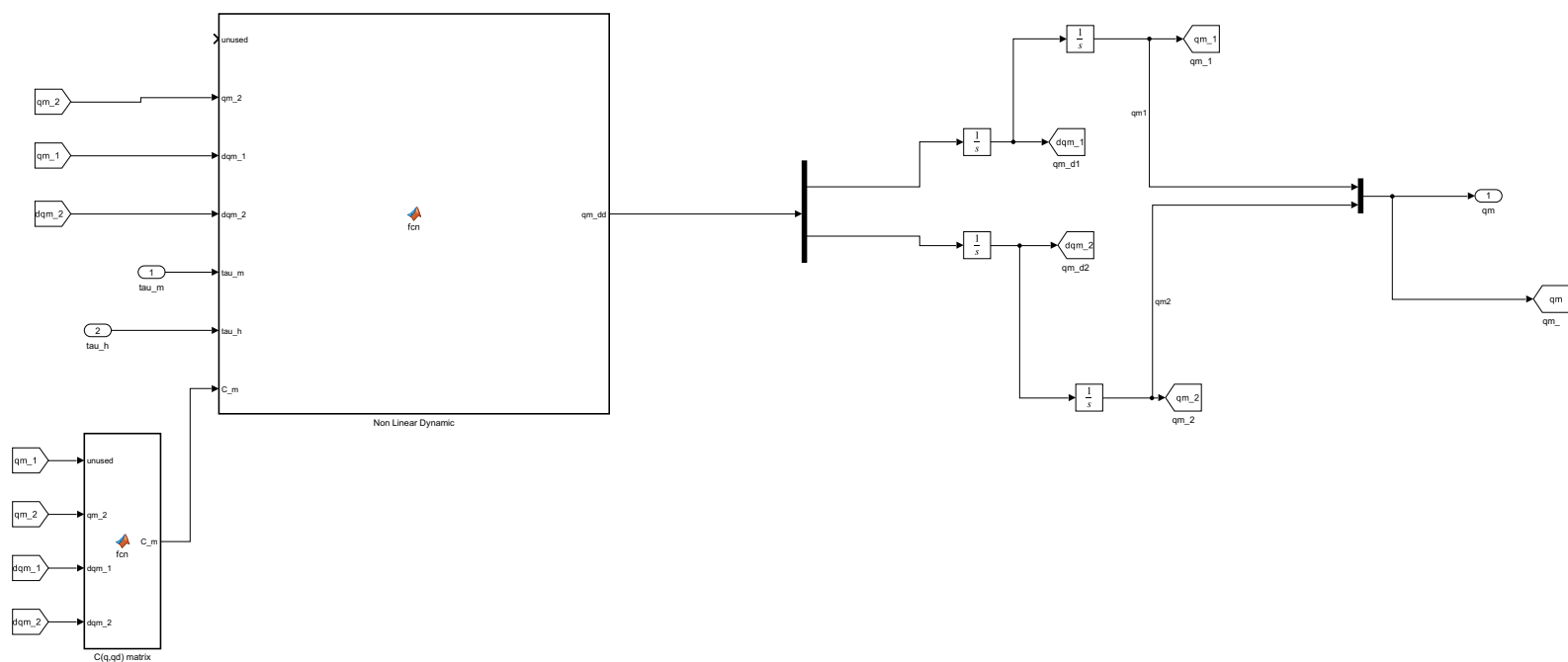






```
function tau_e = fcn()
```

```
tau_e=[0;0];
```



```
function C_m = fcn(~,qm_2,dqm_1,dqm_2)
```

```
% Physiscal Parameters
```

```
Lm1 = 0.38;
```

```
Lm2 = 0.38;
```

```
mm1 = 3.5;
```

```
mm2 = 0.5;
```

```
%Master_first_link_length
```

```
%Master_second_link_length
```

```
%Master_first_link_mass
```

```
%Master_second_link_mass
```

```
betha_m=Lm1*Lm2*mm2;
```

```
C_m_11 = -2*betha_m*sin(qm_2)*dqm_2;
```

```
C_m_12 = -betha_m*sin(qm_2)*dqm_2;
```

```
C_m_21 = betha_m*sin(qm_2)*dqm_1;
```

```
C_m_22 = 0;
```

```
C_m = [C_m_11 C_m_12; C_m_21 C_m_22];
```

```
end
```

```
function qm_dd =fcn(~,qm_2,dqm_1,dqm_2,tau_m,tau_h,C_m)
```

```
% Physcal Parameters
```

```
Lm1 = 0.38;
```

```
Lm2 = 0.38;
```

```
mm1 = 3.5;
```

```
mm2 = 0.5;
```

```
%Master_first_link_length
```

```
%Master_second_link_length
```

```
%Master_first_link mass
```

```
%Master_second_link mass
```

```
dqm=[dqm_1;dqm_2];
```

```
alpha_m=(Lm2^2)*mm2+(Lm1^2)*(mm1+mm2);
```

```
betha_m=Lm1*Lm2*mm2;
```

```
delta_m=(Lm2^2)*mm2;
```

```
M_m_11= alpha_m + 2*betha_m*cos(qm_2);
```

```
M_m_12= delta_m + betha_m * cos(qm_2);
```

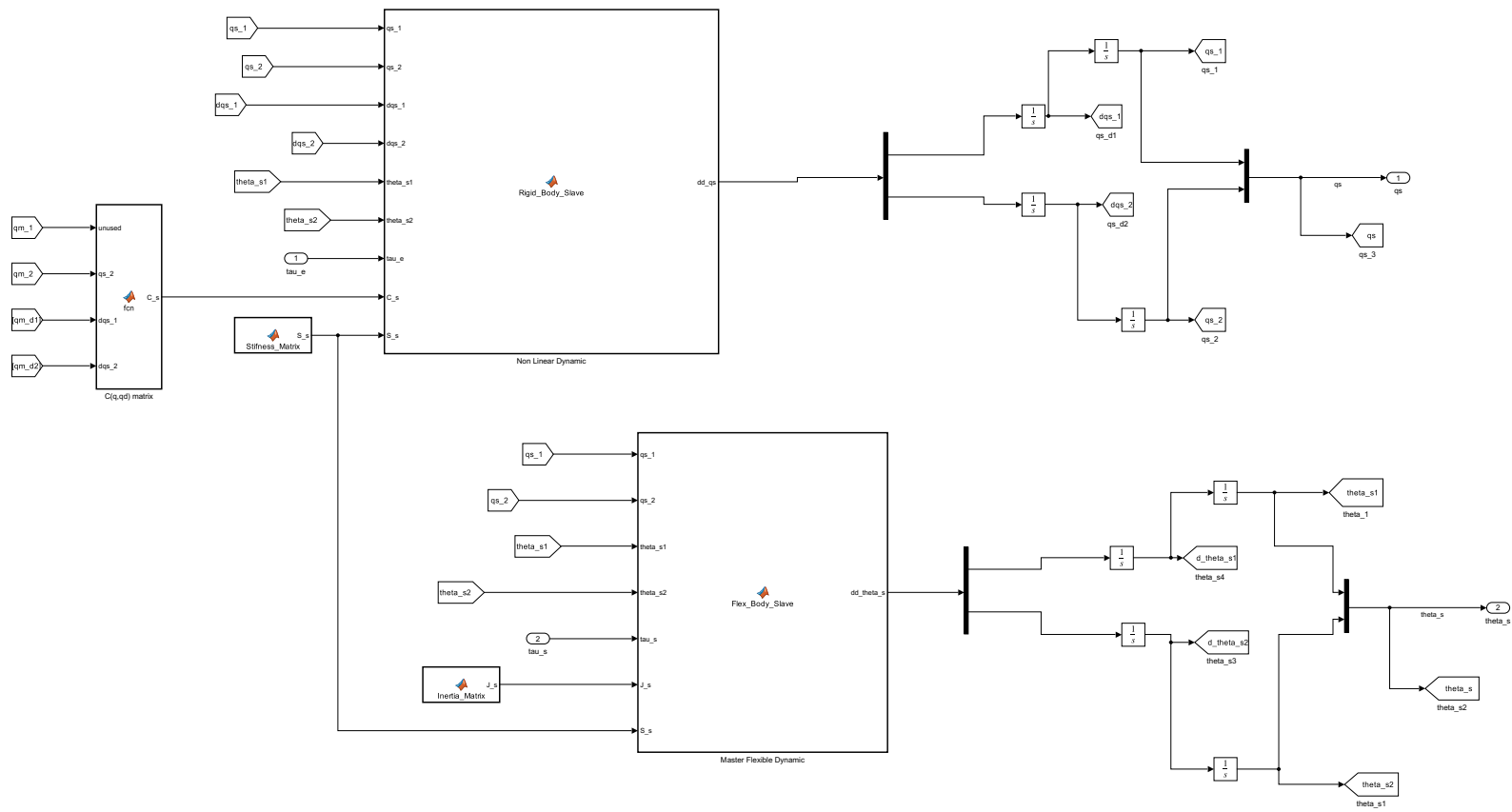
```
M_m_21= M_m_12;
```

```
M_m_22= delta_m;
```

```
M_m = [M_m_11 M_m_12; M_m_21 M_m_22];
```

```
qm_dd = inv(M_m)*( -tau_m -(C_m * dqm ) + tau_h );
```

```
end
```



```

function C_s = fcn(~,qs_2,dqs_1,dqs_2)

% Phyiscal Parameters
Ls1 = 0.3265;           % Link 1 length (m)
Ls2 = 0.3265;           % Link 2 length (m)

ms1 = 0.5362;           % Link 1 mass (kg)
ms2 = 0.5362;           % Link 2 mass (kg)

betha_s=Ls1*Ls2*ms2;

C_s_11 = -2*betha_s*sin(qs_2)*dqs_2;
C_s_12 = -betha_s*sin(qs_2)*dqs_2;
C_s_21 =  betha_s*sin(qs_2)*dqs_1;
C_s_22 =  0;
C_s=[C_s_11 C_s_12; C_s_21 C_s_22];
end

```

```
function S_s = Stifness_Matrix()
    % This function creates a 2x2 identity matrix scaled by 500

    % Create a 2x2 identity matrix
    I = eye(2);

    % Scale the identity matrix by 500
    S_s = 500 * I;
end
```

```
function J_s = Inertia_Matrix()

    % Create a 2x2 identity matrix
    I = eye(2);
    % Scale the identity matrix by 500
    J_s = 0.2 * I;
end
```

```
function dd_theta_s = Flex_Body_Slave(qs_1,qs_2,theta_s1,theta_s2,tau_s,J_s,S_s)

qs=[qs_1;qs_2];
theta_s=[theta_s1;theta_s2];

dd_theta_s = inv(J_s)*(tau_s - S_s*(theta_s-qs));

end
```

```

function dd_qs =Rigid_Body_Slave(qs_1,qs_2,dqs_1,dqs_2,theta_s1,theta_s2,tau_e,C_s,S_s)

%% Physical Parameters (According to the experimental platform)

Ls1 = 0.3265;           % Link 1 length (m)
Ls2 = 0.3265;           % Link 2 length (m)

ms1 = 0.5362;           % Link 1 mass (kg)
ms2 = 0.5362;           % Link 2 mass (kg)

%% State Variables

qs      = [qs_1;qs_2];
theta_s = [theta_s1;theta_s2];
dqs     = [dqs_1;dqs_2];

%% Dynamic Parameters

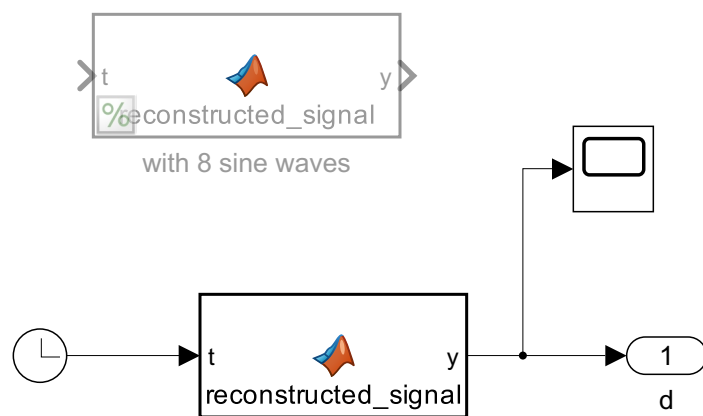
alpha_s = (Ls2^2)*ms2 + (Ls1^2)*(ms1+ms2);
beta_s  = Ls1*Ls2*ms2;
delta_s = (Ls2^2)*ms2;

%% Inertia Matrix

M_s = [ alpha_s + 2*beta_s*cos(qs_2),  delta_s + beta_s*cos(qs_2);
        delta_s + beta_s*cos(qs_2),    delta_s ];

%% Dynamics
dd_qs= inv(M_s)*(- tau_e - (C_s * dqs ) - (S_s*(qs-theta_s)));
end

```



```
function y = reconstructed_signal(t)
% Given data
DC_offset = 0.9613462407597305;
sine_amplitudes = [-0.02000659, -0.02349457, 0.00067474, 0.00159032, 0.00129016, -0.00135477, -0.00160663, -0.00160663];
cosine_amplitudes = [-0.02556397, 0.00393168, -0.0052079, 0.00220769, 0.00300956, 0.00160663, -0.00160663, 0.00160663];
frequencies = [0.24299065416927243, 0.48598130833854486, 0.7289719625078173, 0.9719626166770897, 1.2149352310151716, 1.4579063425232539, 1.7008774540313362, 1.9438485655394185];

% Number of terms
N = length(sine_amplitudes);

% Initialize the signal
y = DC_offset * ones(size(t));

% Add sine and cosine terms
for k = 1:N
    y = y + sine_amplitudes(k) * sin(2 * pi * frequencies(k) * t) + cosine_amplitudes(k) * cos(2 * pi * frequencies(k) * t);
end
end
```

```
function y = reconstructed_signal(t)
% Given data
DC_offset = 0.9613888244716796;
sine_amplitudes = [-0.02002779, -0.02351674, 0.00074362, 0.00154452, 0.0013555, -0.00135695, -0.00135695, -0.00135695];
cosine_amplitudes = [-0.02562599, 0.00393932, -0.00523797, 0.0022634, 0.00295895, 0.00158273, -0.00158273, -0.00158273];
frequencies = [0.24299065416927243, 0.48598130833854486, 0.7289719625078173, 0.9719626166770897, 1.2149352331493794, 1.457907394771469, 1.700879556393558, 1.943851718015648];

% Number of terms
N = length(sine_amplitudes);

% Initialize the signal
y = DC_offset * ones(size(t));

% Add sine and cosine terms
for k = 1:N
    y = y + sine_amplitudes(k) * sin(2 * pi * frequencies(k) * t) + cosine_amplitudes(k) * cos(2 * pi * frequencies(k) * t);
end
end
```

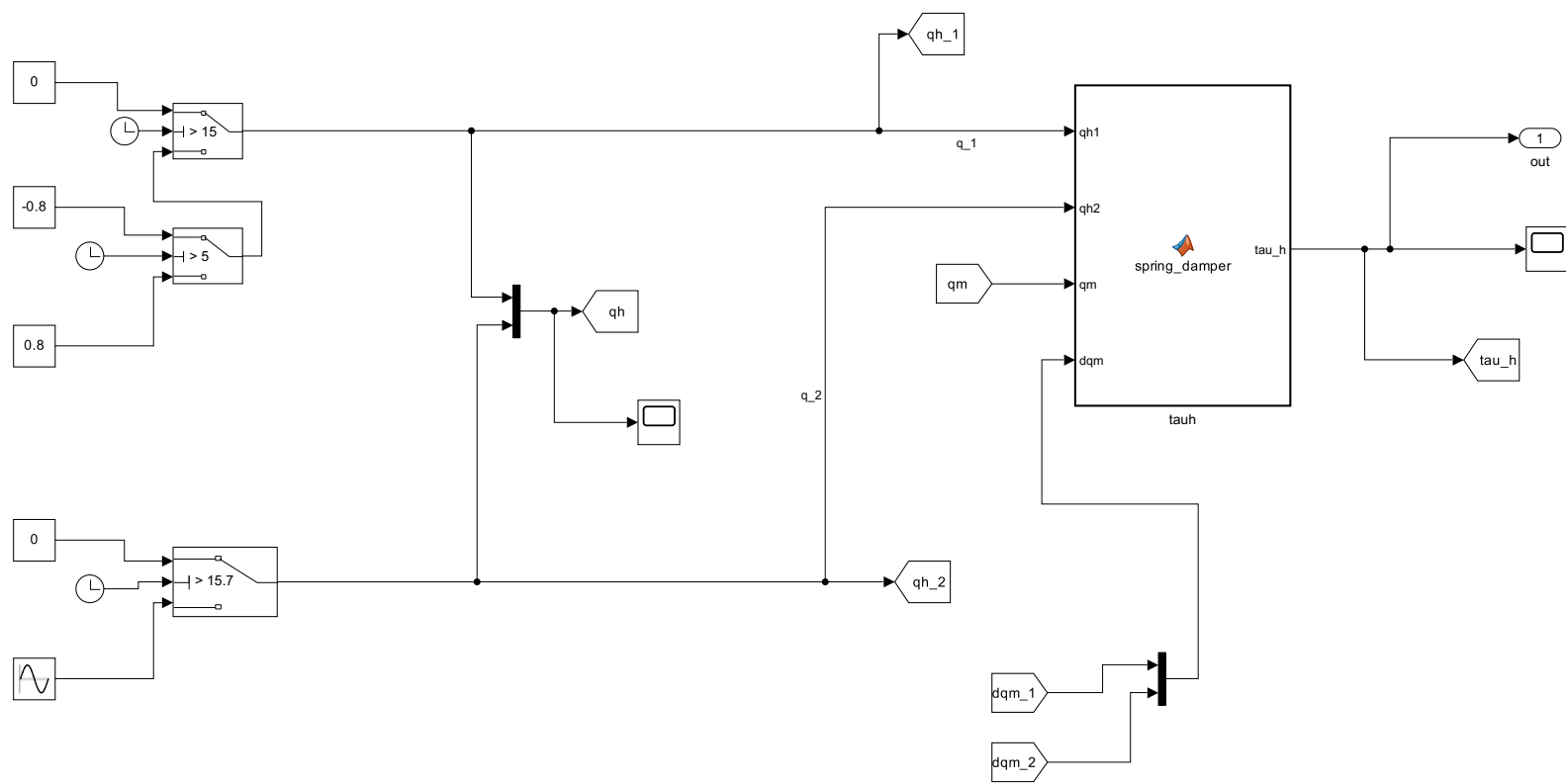


```
function y = reconstructed_signal(t)
% Given data
DC_offset = 0.9613462407597305;
sine_amplitudes = [-0.02000659, -0.02349457, 0.00067474, 0.00159032, 0.00129016, -0.00135477, -0.00160663, -0.00160663];
cosine_amplitudes = [-0.02556397, 0.00393168, -0.0052079, 0.00220769, 0.00300956, 0.00160663, -0.00160663, 0.00160663];
frequencies = [0.24299065416927243, 0.48598130833854486, 0.7289719625078173, 0.9719626166770897, 1.2149352310151716, 1.4569063425232539, 1.6988774540313362, 1.9408485655394185];

% Number of terms
N = length(sine_amplitudes);

% Initialize the signal
y = DC_offset * ones(size(t));

% Add sine and cosine terms
for k = 1:N
    y = y + sine_amplitudes(k) * sin(2 * pi * frequencies(k) * t) + cosine_amplitudes(k) * cos(2 * pi * frequencies(k) * t);
end
end
```



```
function tau_h = spring_damper(qh1,qh2,qm,dqm)
```

```
Kh=10;
```

```
Dh=2;
```

```
qh=[qh1;qh2];
```

```
tau_h=Kh*(qh-qm)-Dh*dqm;
```

```
end
```